Design and Implementation of Distributed Time-Triggered System Based on RT-Thread

Chunmeng Zhong
School of Software, Tsinghua University
Beijing, China
+86 18340806466
18340806466@163.com

Haifeng Zhang
Beijing Smart-Chip Microelectronics Technology Co., Ltd.
zhanghaifeng@sgitg.sgcc.com.cn

Yidong Yuan
Beijing Smart-Chip Microelectronics Technology Co., Ltd.
yuanyidong@sgitg.sgcc.com.cn

Sukun Zhang
School of Software, Tsinghua University
Beijing, China
zsk18@mails.tsinghua.edu.cn

Hai Wan
School of Software, Tsinghua University
Beijing, China
wanhai@tsinghua.edu.cn

ABSTRACT
Internet of Things (IoT) technologies are driving changes in Industry 4.0. Applications developed based on the IoT system may have high requirements on the real-time property of data transmission. To provide the real-time data transmission service for time-sensitive applications, Time-Sensitive Networking (TSN) standards are proposed. Though TSN standards have relatively perfect support for real-time transmission at the switch level, it pays little attention to the real-time guarantee of end devices. In order to ensure the real-time behavior of end systems, a time-triggered thread-level scheduling mechanism is devised. Based on this scheduling mechanism, time sensitive tasks, implemented as threads, are executed periodically at predefined times according to a global notion of time, which as a result can ensure the end to end delay of a message. Design and implementation of this mechanism are completed based on RT-Thread, a widely used lightweight embedded real-time operating system. Comprehensive test and experiments are conducted to show the effectiveness of the method proposed.

CCS Concepts
• Computer systems organization→Real-time systems • Computer systems organization→Embedded and cyber-physical systems • Security and privacy →Systems security.

Keywords
Real-time; time-triggered; distributed system.

1. INTRODUCTION
IoT technologies are driving changes in Industry 4.0, known as the fourth industrial revolution [2]. The IoT system is always composed of distributed heterogeneous devices, for example, sensors, actuators and processors [3]. Some user applications developed based on IoT system have high requirements on real-time data transmission, such as smart city systems used to manage traffics [4]. The datagram among these applications need to arrive at destination device at predefined times [5]. Some other applications based on distributed devices are also time sensitive, such as iSpace system which is used to promote factory production [6]. These time-sensitive applications require real-time message transmission services to work properly, including real-time message sent from the end device and real-time message transmission in the network.

To provide the real-time message transmission service for time-sensitive applications, IEEE 802.1 group developed the TSN standards. These standards define mechanisms for the time-sensitive data transmissions over deterministic Ethernet networks. TSN improves the determinacy of traditional Ethernet, enabling the IoT to evolve into the real-time IoT. In applications with high real-time requirements, such as robotic arms and motion controllers [7], TSN can be used to synchronize networks of devices to improve the deterministic behavior of industrial systems [1]. TSN standards has relatively perfect support for real-time transmission in networks, but pays little attention to the real-time performance of end devices. P802.1 as-rev standard is for time synchronization, 802.1Qbu standard is for frame preemption, and 802.1Qbv standard is for flow scheduling, other protocols also rarely involve end devices.

However, end device is an important link in the data transmission chain among distributed devices, which is the starting point and end points of the transmission. To enable real time transmission on end device, two problems need to be solved. First, a concept of global time should be setup. The sending times of different devices need to be orchestrated to eliminate conflicts. It can be solved by using existing time synchronization standard such as IEEE P802.1 as. Second, a time-based scheduling mechanism is needed to synchronize the task execution based on the global time. This mechanism requires modifications of the operating system kernel so that data transmission and task scheduling are executed at a specified time.

ARNIC 653 has given the requirements and definitions for time-based scheduling. VxWorks, SylixOS and other operating systems have implemented ARNIC 653 requirements. But there are two problems with these implementations. First, the operating system is

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
ICIT 2019, December 20–23, 2019, Shanghai, China
© 2019 Association for Computing Machinery.
ACM ISBN 978-1-4503-7663-1/19/12…$15.00
DOI: https://doi.org/10.1145/3377170.3377268

somehow heavy. IoT systems typically have a large number of end devices [8]. Hence, end devices should be low-cost, which mean the computation power and cost are low. Therefore, OS and applications on the end device need to be lightweight. Second, the time-triggered executions defined in ARNIC 653 is specific to the partition, and the scheduling of tasks within the partition is uncertain (for example, priority based scheduling). Only time-based scheduling at thread-level can enable real-time data transmission on end devices. A time-based scheduling system that meets both lightweight and thread-level needs to be studied. RT-Thread is a widely used lightweight embedded real-time operating system. It is stable and reliable, with low requirements on computing resources. In this paper, we design and implement a time-triggered (TT for short) thread-level scheduling based on RT-Thread, which ensures that a task is triggered to run at predefined times and is an effective method to complement the defect of real - time data transmission chain.

There are two major contributions of this paper. First, we implemented a TT thread-level scheduling extension based on RT-Thread. The threads in RT-Thread are divided into time-sensitive threads and non-time-sensitive threads. Time-sensitive threads can be scheduled periodically at predefined times. Second, time-triggered scheduling based on global clock. The global clock needs not be consistent with the local clock. The process of message data transmission in the end devices is optimized in such a way that the data transmission is not affected by queuing delay or other factors, and the data transmission is guaranteed to be real-time and deterministic.

The remainder of this paper is organized as follows. After introducing the need for TT threads, we cover them in detail in Section 2. Section 3 and Section 4 describe the design and implementation of TT thread extension. Section 5 is about the testing of TT thread extension. The relevant work is in Section 6. Section 7 gives the summary and the future prospect.

2. RELATED WORK

VxWorks 653 implemented application-level time synchronization between different devices. But its time-triggered scheduling is specific to the partition, and the scheduling of tasks within the partition is uncertain. [9] expresses periodic time-triggered tasks on multiple processing nodes as a Mixed Integer Programming problem formally but it lacks the discussion of the thread-level time-triggered case and it doesn't provide an implementation of the operating system.

In [10], TT thread extension is implemented based on the NEC V850 platform, but the number of threads is configured statically before compilation which leads to inefficient resource utilization. Other researchers implement the timer-trigger task scheduling and periodic-trigger task scheduling respectively based on the NEC V850 platform but the period of time-triggered task scheduling implemented in this paper takes 4 milliseconds as the basic unit, which is not applicable in the high-precision industry.

3. DISTRIBUTED TT SYSTEM

Time sensitive applications depends on the real-time performance of data transmission service, such as traffic management, aerospace, weapons, etc. If message is not delivered at the expected time, fatal errors would occur [5]. The so-called distributed TT system composes of TSN-switches and end devices. End systems exchange time-sensitive and non-time-sensitive data using TSN switches. Once data arrives at the switches, the network made up of TSN switches can guarantee the real-time performance of the data transmission. However, the real-time guarantee between the task (which is the producer and consumer of the data) and the TSN-switch is not ensured. In order to make the data sending time of end device align to the receiving time of the TSN switch, we proposed a time-triggered task level scheduling mechanism. Time-sensitive tasks are realized as TT threads, which are executed periodically at predefined times.

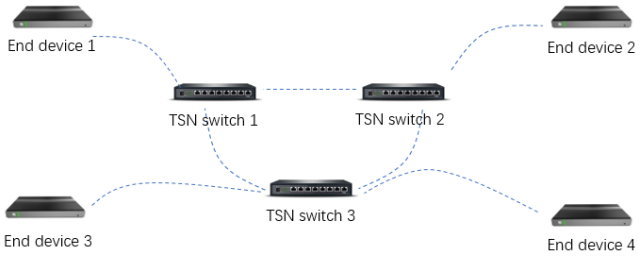


Figure 1. A typical topology of distributed TT system consists of end devices and TSN switches

In order to provide deterministic TT thread scheduling service, the concept of a global time is needed. A TT thread may need to send and receive data. In order to improve the real-time performance of data transmission between the thread and the network interface card, two queues are devised in the NIC driver and the NIC FPGA code. One queue is used to store non-time-sensitive message and the other is used to store time-sensitive message. The TT thread will put time-sensitive message into the time-sensitive queue. How to synchronize time and design and implement two queues is out of the scope of the paper. In this paper, we focus our discussion on the design and implementation of the TT scheduling mechanism based on a specific real-time OS.

4. SYSTEM DESIGN

In this section, we mainly introduce the TT thread expansion requirements and implementation ideas. Due to the particularity of the TT thread, it is necessary to design special management, maintenance, and scheduling logic for it. Including the whole life cycle of threads from creation to destruction, methods of thread storage and maintenance, and time-triggered scheduling of TT threads.

4.1 Thread Management

The management of TT threads runs through all phases of the life cycle, including creation, startup, suspension, scheduling, and destruction. Thread management is the foundation of the TT thread to complete its expected function, and it is also an important method of modularization.

4.1.1 Thread Startup

We need to add TT threads to ready list to startup them. TT threads are scheduled by time so there is no need to call thread switching after TT threads are added to the ready queue. Besides, ET threads need to record the number of the corresponding priority thread, while TT threads only need to record the number of TT threads.

4.1.2 Thread Scheduling

As shown in Figure 2, TT threads and ET threads share a common thread scheduling method. TT threads should have a higher priority to all ET threads, so the TT threads trigger condition should be detected first when switching threads. If the trigger condition of TT threads is satisfied, then the ET thread will switch to the corresponding TT thread. It is important to note that, during the execution of the scheduling process, if one TT thread is executing currently and the TT thread has not completed its tasks of this

period, the TT thread will not be scheduled and the TT thread continues to execute.

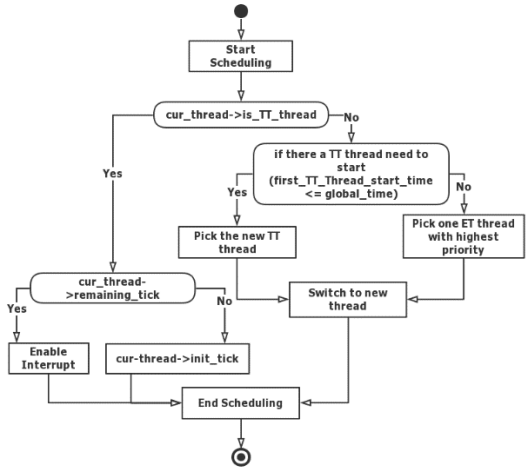


Figure 2. TT thread scheduling.

4.1.3 Thread Removal

If a TT thread is no longer needed or if a TT thread has an exception during execution, the TT thread needs to be killed and removed from the ready list and freeing up space. When deleting, it should not affect the running of the ET thread, and a thread switch function should be called immediately.

4.2 Time-Triggered Scheduling

TT threads are scheduled based on time and have the higher priority than ET threads, and the task execution time is controllable. The trigger logic for TT threads is shown in Figure 3. The system uses hardware clock interrupt to monitor time. When the TT thread execution time arrives, the thread switching module is triggered. To implement global clock synchronization, we use the global (external) time to determine whether TT thread execution time

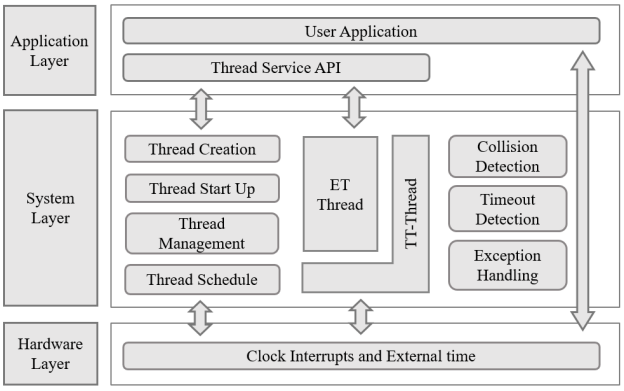


Figure 3. TT thread structure.

arrives. Only by using external time can all devices work properly because the local time at different devices is difficult to be completely consistent.

4.3 Schedule Scenarios and Method

TT threads must execute at a predefined time, while ET threads have no strict time requirements. TT threads must be scheduled before ET threads. There are four possible scenarios of switching between threads:

A. *Switch from ET thread to TT thread*

If it arrives at the TT thread trigger time, the ET thread is suspended and the TT thread occupies the CPU. After the ET threads are suspended, they are put at the head of the ET thread ready list.

B. *Switch from TT thread to ET thread*

The TT thread completes its task of this period and then suspends itself. In this case, the TT thread will be suspended and the highest priority ET thread occupies the CPU.

C. *Switch from ET thread to ET thread*

ET thread switching triggered by system events. TT threads can also be triggered by system events. To avoid the conflict between event-triggered switching and time-triggered switching, special processing is done in the scheduling function.

D. *Switch from TT thread to TT thread*

This is a special case that only happens when the TT thread suspends itself at the execution time of the next TT thread. In this case, the new TT thread occupies the CPU.

Besides, if a TT thread timeout exception occurs, thread suspension and switch are also triggered. Although the process is different, the result is the same as that of scenario B and scenario D.

4.4 Time Isolation Between TT Threads

Since TT threads must run at a specified time and cannot be interrupted, one period can only be occupied by at most one TT thread. To indicate the period, the TT thread must declare its trigger time and maximum execution time.

Use P represents the period of TT thread, F represents the offset within the period, and S represents the maximum execution time. Multiple TT threads may exist simultaneously when the system running, one of which can be formally expressed as:

$$\mathrm{W_i} = \{ P_i, F_i, S_i \}$$

With k as the number of period, the trigger time H of a TT thread can be expressed as:

$$\mathrm{H_i} = P_i \times k_i + F_i$$

Therefore, the current time period declared by TT thread within one cycle is $[\mathrm{H_i}, \mathrm{H_i} + S_i)$, and this time period owned by itself alone. This rule must not be violated during all the periods before it finishes running.

TT threads will not be affected by ET threads because the priority of TT thread is strictly higher than that of ET threads. ET threads can only execute during periods when no TT thread is running. That is to say, there is time isolation between ET and TT threads. Time isolation is an important means to ensure the security of the operating system [12]. Time isolation between TT threads and ET threads, as well as between TT threads, guarantees the security and reliability of user critical tasks.

5. SYSTEM IMPLEMENTATION

We implemented the TT Thread expansion based on RT-Thread lts-v3.1.3 version. Lts v3.1.3 version is a permanent maintenance version of embedded real-time operating system RT-Thread.

5.1 Linked List Maintenance

TT threads adopt the time-triggered scheduling strategy. The system should find the thread with the earliest execution time when a thread scheduling is triggered. One of the best strategies is to maintain an ordered list of threads. The difficulty of this strategy is

to keep the list orderly after each addition with minimum time complexity.

Adding a node to the TT thread list is the longest and uninterruptible operation in the entire TT thread scheduling process. We use the SkipList structure to add nodes with O(log N) time complexity [13].

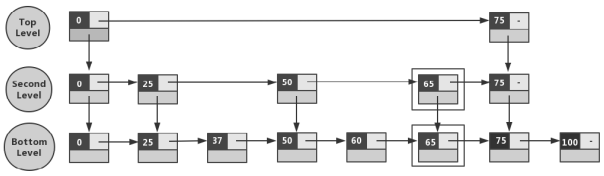


Figure 4. Find the insertion position and add a node to Skip List.

As shown in Figure 4, the SkipList is a multilevel linked list. Each TT thread has an array of linked list nodes. The nodes in the array will be inserted into the corresponding layer of the SkipList. The SkipList requires only the complexity of O(log N) to find node insertion position, which can greatly reduce the time complexity.

Due to the multi-level linked list structure of SkipList, it is necessary to find the insertion position in every layer. We can use a list node array to store the insertion position. From top to bottom, traverse all the layers of SkipList. Use *ROW-LVL* represents the layer number and *ROW-HEAD* is a list node array. In the *ROW-LVL* layer, traverse all nodes to find the correct insertion position and then record it in *ROW-HEAD[ROW-LVL]*.

Then insert the nodes of thread skip list nodes to the corresponding SkipList layer. TT thread node is not necessarily added to any layers except the bottom layer. From the bottom to the top, each layer has a probability of 0.5 to add nodes. If one layer does not add the current TT thread node, then all layers above it do not add the node. We use random numbers to implement the adding probability.

5.2 TT Thread Schedule

Section 3.3 analyzes four scenarios of thread scheduling, in which TT thread switching to ET thread and ET thread switching to ET thread can directly use the original thread scheduling logic.

ET thread switching to TT thread and TT thread switching to TT thread need to take precedence over other two scenarios. During thread scheduling, we should first judge if the current global time has reached the earliest TT thread execution time. If so, then switch from the current thread to the first TT thread in the TT thread ready list, which has the earliest execution time.

If the current TT thread has not completed the task within this period when the thread switches, ET threads cannot interrupt the TT thread and the current TT thread continues to execute.

5.3 Timeout Detection

As described in section 3.4, the TT thread must declare its own maximum execution time to avoid time conflict with other threads.

As shown in Figure 5, if TT thread *A* times out, it will affect the execution of TT thread *B*. We use the time slice parameter to implement timeout detection. Users can set the TT thread handling methods after the timeout, such as printing error messages.

5.4 Time Collision Detection

As discussed in section 3.4, one period can only be occupied by one TT thread. As shown in Figure 5, TT thread *B* and TT thread *C* have a time conflict, resulting in the execution of TT thread *B* being

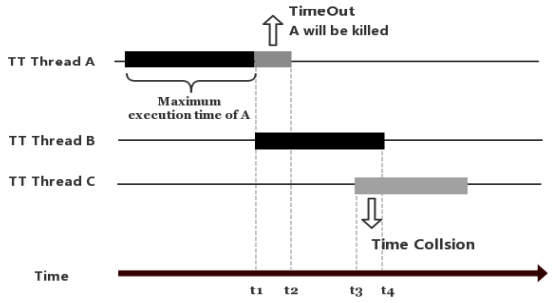


Figure 5. TT thread timeout (TT thread A) and Time Collision (TT thread C).

interrupted by TT thread *C*, which is intolerable. The time occupied by a TT thread is determined by three factors, the execution period *P*, offset *F* and the maximum execution time *S*. For two TT threads *i* and *j*, the sufficient and necessary conditions that they do not conflict in time are as follows:

$$\forall\ k_i, k_j \in Z:\ H_i - H_j > S_j\ \vee\ H_j - H_i > S_i$$

in witch

$$H_i = F_i + K_i * P_i\ \vee$$
$$H_j = F_j + K_j * P_j$$

Due to the periodicity of the data stream, the strictest form of the above equation can be obtained by the modular operation.

$$(F_i - F_j) \bmod \gcd(P_i, P_j) > S_j\ \vee$$
$$(F_j - F_i) \bmod \gcd(P_i, P_j) > S_i$$

gcd(a,b) is the greatest common divisor of the integer a, b. It can

be seen that, as long as all created TT threads meet the above conditions, they will not occupy the same time in all periods, that is, there will be no time conflicts.

6. EVALUATION AND ANALYSIS

Series of tests of RT-Thread TT Thread extension have been executed using *STM32H743* hardware platform and *STM32H743-STK-APOLLO* RT-Thread official project.

6.1 Correct Test

In this test, 50 TT threads were created and started. Their period is 6000ms. Their offset is a random number less than 6000, and the maximum execution time is set to 10. The entry function is an infinite loop function.

Table 1. Correct test configuration

index	type	period	offset	exec time	describe
1-50	TT	6000	random	10	Inf

Table 2. Correct test result

test time	exec count	creation count	max running	error
1.5h	46475	50	50	None

As can be seen from the test results, TT threads perform expected functions correctly.

6.2 Real Environment Test

In this test, 50 TT threads were randomly created with the same attributes as the test in section 5.1. There is also an ET thread. The

ET thread entry function sleeps for 100 milliseconds after completing one loop.

Table 3. Real environment test configuration

index	type	period	offset	exec time	describe
0-50	TT	6000	random	10	Inf
51	ET	-	-	-	Inf

Table 4. Real environment test result

test time	exec count	creation count	max running	error
5h	145818	50	50	None

As can be seen from the test results, TT thread can still perform expected functions correctly under the interference of the ET thread.

6.3 Pressure Test

In pressure test, hundreds of TT threads were created throughout the test. The number of TT threads is only limited to the system storage space. At the same time, 10 ET threads with infinite loop entry functions are running in the system together with TT threads. The 10 ET threads fall into two categories. Five of which have a priority of 24 and are suspended 50 milliseconds after completion of each loop, and the other five have a priority of 25 and are suspended 500 milliseconds after completion of each loop.

Table 5. Pressure test configuration

index	type	period	offset	exec time	describe
1-5	ET(priority 25)	-	-		Inf
6-10	ET(priority 24)	-	-	-	Inf
Inf	TT	6000	Random	10	Fin

Table 6. Pressure test result

test time	exec count	creation count	max running	error
17h	2759182	136043	277	None
18h	2885174	142532	179	None

As can be seen from the test results, the system can still operate normally and the TT threads can complete its function correctly.

7. SUMMARY AND CONCLUSIONS

RT-Thread TT Thread extension provides a thread-level time-triggered service for end devices. Combined with TSN, it provides perfect real-time data transmission for distributed system. But TSN rarely involves real-time performance on the end devices, while TT thread expansion complements this defect.

Compared with the existing time-triggered system, our implementation has the following advantages. First, TT threads can be added dynamically. Second, thread stack size can be allocated as needed. So memory resources are utilized to the fullest extent. Third, thread execution time are controllable. TT threads can either loop indefinitely or stop automatically after a finite number of executions. Most importantly, TT thread extensions support global time synchronization, which is very suitable for distributed time-triggered systems.

At present, RT-Thread time-triggered accuracy is 1 millisecond and it still needs to be further improved to adapt to more stringent industrial control environment. Moreover, due to the influence of interrupt shielding in the system, it is difficult to guarantee the periodicity of the system clock. We use an external clock to alleviate the time drift, but the real-time performance will still be affected by interrupt shielding.

8. ACKNOWLEDGMENTS

This research is sponsored by “Key technology and application research of security MCU-chip for intelligent electrical terminal” project (No.546816180009) of State Grid Corporation of China.

9. REFERENCES

[1] “Adopting time-sensitive networking (tsn) for automation systems.” Platform,https://software.intel.com/en-us/articles/adopting-time-sensitive-networking-tsn-for-automation-systems.

[2] L. Atzori, A. Iera, and G. Morabito, “The internet of things: A survey,” Computer Networks, vol. 54, no. 15, pp. 2787 – 2805, 2010.

[3] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, “Internet of things for smart cities,” IEEE Internet of Things Journal, vol. 1, no. 1, pp. 22–32, 2014.

[4] D. Omer, R. Aburukba, and T. Landolsi, “Optimization model for time sensitive iot requests,” in 2019 International Conference on Communications, Signal Processing, and their Applications (ICCSPA), pp. 1–4, March 2019.

[5] M. S. Mahmoud and M. Sabih, “Networked event-triggered control: an introduction and research trends,” International Journal of General Systems, vol. 43, no. 8, pp. 810–827, 2014.

[6] Wai-Lun Danny Leung, R. Vanijjirattikhan, Zheng Li, Le Xu, T. Richards, B. Ayhan, and Mo-Yuen Chow, “Intelligent space with time sensitive applications,” in Proceedings, 2005 IEEE/ASME International Conference on Advanced Intelligent Mechatronics., pp. 1413–1418, July 2005.

[7] “Tsn: Converging networks for a better industrial iot.” https://www.electronicdesign.com/industrial-automation/tsn-converging-networks-better-industrial-iot.

[8] K. E. Skouby and P. Lynggaard, “Smart home and smart city solutions enabled by 5g, iot, aai and cot services,” in 2014 International Conference on Contemporary Computing and Informatics (IC3I), pp. 874–878, Nov 2014.

[9] L. Zhang, D. Goswami, R. Schneider, and S. Chakraborty, “Task- and network-level schedule co-synthesis of ethernet-based time-triggered systems,” in Asia South Pacific Design Automation Conference, 2014.

[10] L. e. HuangShujuan, ZhuYi’an, “Implementation of uc/os-ii kernalfor dou-ble-trigger mechanism,” Computer Measurement Con-trol,vol. v.26No.238, no. 7, pp. 203–206, 2018.

[11] O. Xiangzhen, Z. Yian, L. I. Lian, and S. Xianchen, “A safetycritical embedded real time operating system kernel design,” Computer Engineering, 2019.

[12] S. S. Craciunas, C. M. Kirsch, H. Payer, H. Rock, and A. Sokolova, ¨ Programmable temporal isolation in real-time and embedded execution environments. 2009.

[13] J. Linden and B. Jonsson, “A skiplist-based concurrent priority queue ´with minimal memory contention,” in International Conference on Principles of Distributed Systems, 2013.